

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Разработка кроссплатформенных приложений»

на тему «Работа с файлами»

Вариант №10

Выполнили: ст. гр. 23ВВИ1

Галкин А.П.

Казаков А.Д.

Приняли:

к.т.н. доцент Юрова О.В.

к.т.н. доцент Карамышева Н.С.

Пенза 2026

Цель работы: изучить работу с файлами и механизмы сериализации данных.

Задание:

Модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла (JFileChooser). Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:

1. Класс RecIntegral модифицирован для реализации интерфейса java.io.Serializable, что позволяет сохранять и восстанавливать объекты коллекции в бинарном виде.
2. В графический интерфейс добавлены элементы меню (кнопки) для четырёх операций: сохранить в .txt, загрузить из .txt, сохранить в .bin, загрузить из .bin.
3. Для текстового формата реализована запись/чтение через FileWriter и BufferedReader. Данные записываются построчно с разделением пробелами, при чтении строка разбивается методом split() и парсится в double.
4. Для бинарного формата использованы ObjectOutputStream и ObjectInputStream. Коллекция LinkedList<RecIntegral> сохраняется и восстанавливается целиком как единый сериализуемый объект.
5. Во всех операциях используется стандартный компонент JFileChooser с фильтрами расширений (.txt и .bin). Обработка ошибок ввода-вывода реализована через блоки try-catch с выводом JOptionPane.

Листинг программы:

Фрагмент RecIntegral.java (реализация Serializable):

```
package pkg23vvv1;

import java.io.Serializable;

public class RecIntegral implements Serializable {

    private static final long serialVersionUID = 1L;
```

```

private static final double MIN_VALUE = 0.00000001;
private static final double MAX_VALUE = 100000000;
private double upperLim;
private double lowLim;
private double step;
private double result;

// ... конструкторы, геттеры, валидация и метод calculate оставлены без изменений ...
}

```

Фрагмент JForm.java (обработчики сохранения/загрузки):

```

private void jMenuItem1SaveTxtActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser f = new JFileChooser();
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text file (*.txt)", "txt");
    f.setFileFilter(filter);
    f.showSaveDialog(null);
    File file = f.getSelectedFile();
    if (!file.getPath().endsWith(".txt")){
        file = new File(file.getPath() + ".txt");
    }
    try(FileWriter writer = new FileWriter(file, false)){
        for (RecIntegral o : tableContent){
            writer.write(o.GetLowLim() + " " + o.GetUpperLim() + " " + o.GetStep() + " " +
o.GetResult() + "\n");
        }
        writer.flush();
        writer.close();
    } catch(IOException ex){

```

```
        JOptionPane.showMessageDialog(null, ex);
    }
}
```

```
private void jMenuItem2LoadTxtActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser f = new JFileChooser();
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text file (*.txt)", "txt");
    f.setFileFilter(filter);
    int result = f.showOpenDialog(null);
    if (result != JFileChooser.APPROVE_OPTION) return;
    File file = f.getSelectedFile();
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        tableContent.clear();
        String temp;
        while ((temp = reader.readLine()) != null) {
            temp = temp.trim();
            if (temp.isEmpty()) continue;
            String[] parts = temp.split(" ");
            if (parts.length != 4) continue;
            RecIntegral rec = new RecIntegral(
                Double.parseDouble(parts[0]), Double.parseDouble(parts[1]),
                Double.parseDouble(parts[2]), Double.parseDouble(parts[3]));
            tableContent.add(rec);
        }
        DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
        model.setRowCount(0);
    }
}
```

```

        for (RecIntegral obj : tableContent) {
            model.addRow(new Object[]{obj.GetLowLim(), obj.GetUpperLim(), obj.GetStep(),
obj.GetResult()});
        }

        JOptionPane.showMessageDialog(null, "Successfully loaded " + tableContent.size() + "
records");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(null, "Error reading file: " + ex.getMessage());
    }
}

```

```

private void jMenuItem3SaveBinActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser f = new JFileChooser();

    FileNameExtensionFilter filter = new FileNameExtensionFilter("Binary file (*.bin)", "bin");
    f.setFileFilter(filter);
    f.showSaveDialog(null);

    File file = f.getSelectedFile();
    if(!file.getPath().endsWith(".bin")){
        file = new File(file.getPath() + ".bin");
    }

    try(ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file)){
        out.writeObject(tableContent);

        out.flush();

        out.close();
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(null, "Error reading file: " + ex.getMessage());
    }
}

```

```
}
```

```
private void jMenuItem4LoadBinActionPerformed(java.awt.event.ActionEvent evt) {  
    JFileChooser f = new JFileChooser();  
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Binary file (*.bin)", "bin");  
    f.setFileFilter(filter);  
    int result = f.showOpenDialog(null);  
    if (result != JFileChooser.APPROVE_OPTION) return;  
    File file = f.getSelectedFile();  
    try (ObjectInputStream in = new ObjectInputStream(new BufferedInputStream(new  
FileInputStream(file)))) {  
        tableContent = (LinkedList<RecIntegral>) in.readObject();  
        DefaultTableModel model = (DefaultTableModel) jTable1.getModel();  
        model.setRowCount(0);  
        for (RecIntegral obj : tableContent) {  
            model.addRow(new Object[]{obj.GetLowLim(), obj.GetUpperLim(), obj.GetStep(),  
obj.GetResult()});  
        }  
    } catch (IOException | ClassNotFoundException ex) {  
        JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());  
    }  
}
```

Скриншоты:

Меню

Сохранить как txt

Загрузить как txt

Сохранить как bin

Загрузить как bin

Верхний порог

5

Шаг

0.5

Добавление из коллекции

Очистить таблицу

Добавить

Удалить

Вычислить

Нижний порог	Верхний порог	Шаг	Результат
1	5	0.5	3.3213100130e+01

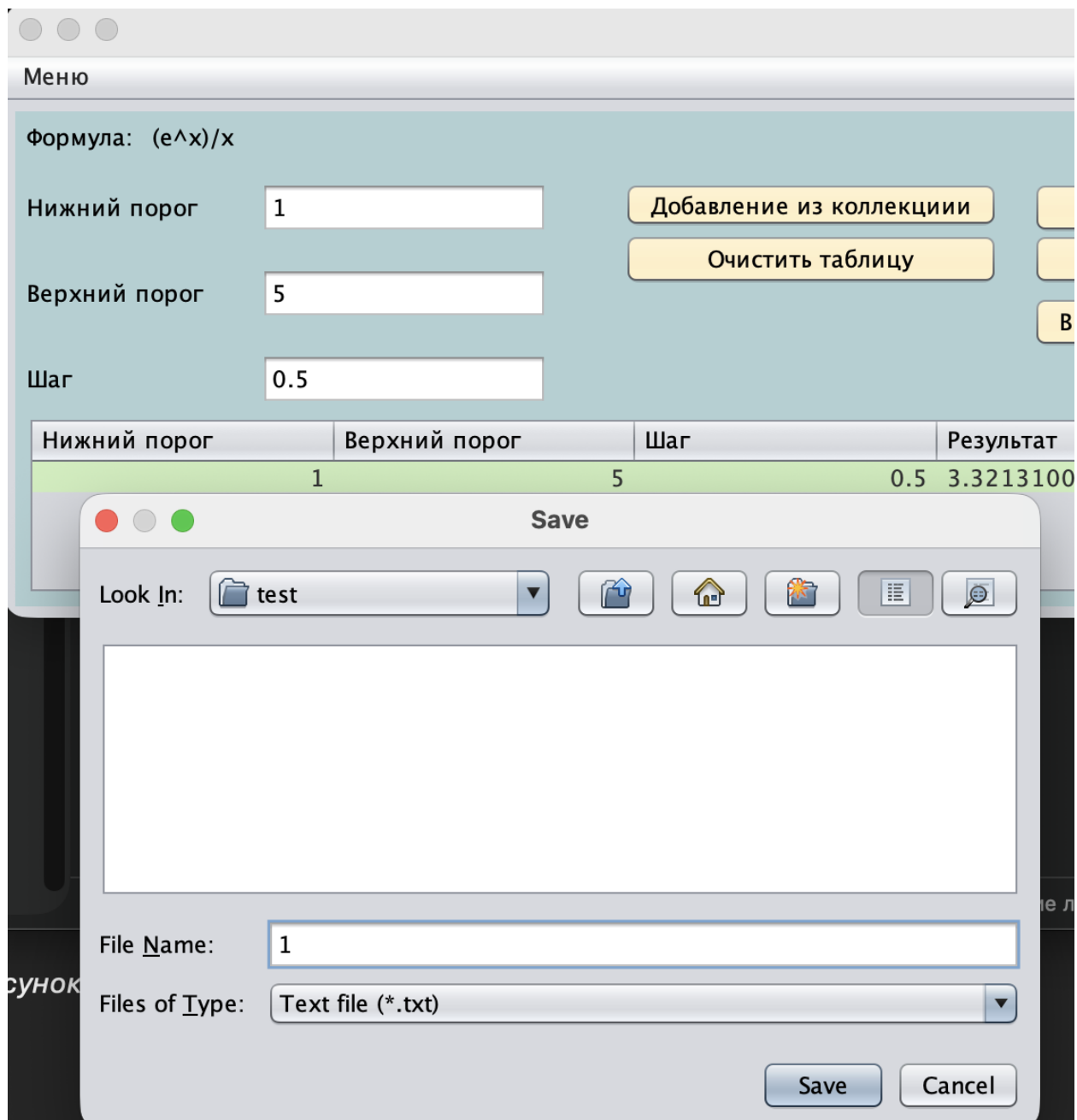


Рисунок 1 – Диалог выбора файла при сохранении (JFileChooser)

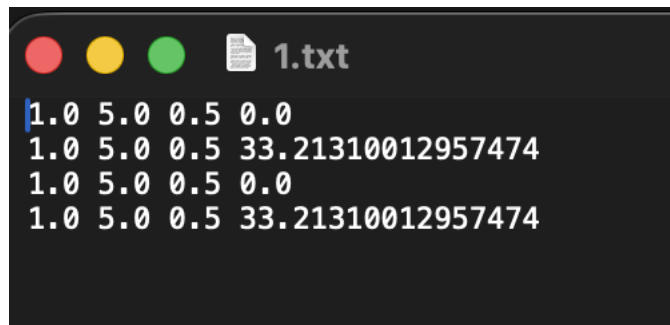
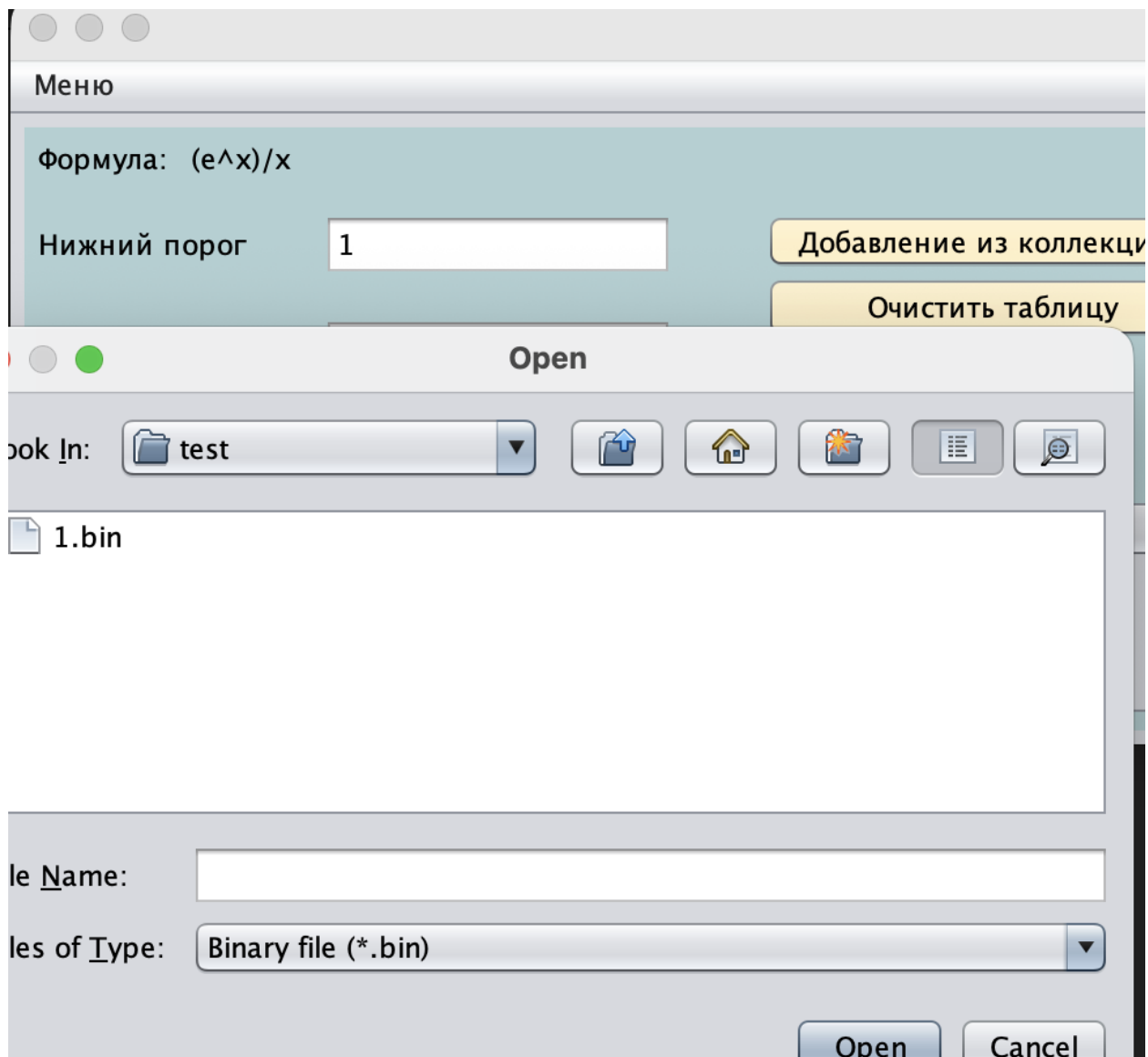


Рисунок 2 – Содержимое сохранённого текстового файла



Формула: $(e^x)/x$

Нижний порог: 1

Верхний порог: 5

Шаг: 0.5

Добавление из коллекции

Добавить

Очистить таблицу

Удалить

Вычислить

Нижний порог	Верхний порог	Шаг	Результат
1	5	0.5	0.0
1	5	0.5	33.21310012957474
1	5	0.5	0.0
1	5	0.5	33.21310012957474

Рисунок 3 – Успешная загрузка данных из бинарного файла

Вывод: в ходе выполнения лабораторной работы изучены механизмы работы с файлами в Java. Реализовано сохранение и загрузка данных таблицы в текстовом формате с использованием `FileWriter/BufferedReader` и в двоичном формате с помощью `ObjectOutputStream/ObjectInputStream`. Класс `RecIntegral` реализовал интерфейс `Serializable` для корректной сериализации коллекции. Применён стандартный диалог `JFileChooser` для выбора файлов, добавлена обработка исключений ввода-вывода. Получены практические навыки организации постоянного хранения данных и восстановления состояния приложения.